

Proyecto #2 2026: Pintar la valla – Cobertura de intervalos de costo mínimo

Grupo de trabajo

Mayo 2026

1. Descripción del problema

Se modela una valla discreta con posiciones enteras desde 0 hasta L . Cada pintor i cubre un intervalo cerrado $[l_i, r_i]$ y tiene un costo positivo c_i . El objetivo es seleccionar un subconjunto de pintores cuya unión cubra toda la valla y cuyo costo total sea mínimo.

Este problema es una variante clásica de cobertura de intervalos ponderada. Para los experimentos de este proyecto se asume que las coordenadas son enteras, de modo que la programación dinámica puede recorrer la valla posición por posición.

2. Análisis del problema

2.1. Subestructura óptima

Sea $dp[x]$ el costo mínimo para cubrir el prefijo $[0, x]$. Una solución óptima para $[0, x]$ termina con algún intervalo i que cubre la posición x . Si ese intervalo inicia en l_i , entonces todo el prefijo anterior $[0, l_i - 1]$ debe estar cubierto de manera óptima, porque cualquier solución mejor para ese prefijo produciría una solución mejor para $[0, x]$.

Por tanto, el problema presenta subestructura óptima.

2.2. Relación de recurrencia

La recurrencia exacta es:

$$dp[x] = \min_{i: l_i \leq x \leq r_i} (dp[l_i - 1] + c_i),$$

donde $dp[-1] = 0$. Si no existe intervalo que cubra x , entonces $dp[x] = \infty$.

3. Algoritmos de solución

3.1. Programación dinámica

Algoritmo 1 DP exacta para cobertura de la valla

Requerir: Valla de longitud L , intervalos (l_i, r_i, c_i)

Garantizar: Costo mínimo y conjunto de intervalos elegidos

```
1: Inicializar  $dp[0..L] \leftarrow \infty$ 
2: Para  $x \leftarrow 0$  to  $L$  hacer
3:    $mejor \leftarrow \infty$ 
4:   Para cada intervalo  $i$  hacer
5:     Si  $l_i \leq x \leq r_i$  entonces
6:        $prev \leftarrow 0$  si  $l_i = 0$ , en otro caso  $dp[l_i - 1]$ 
7:        $mejor \leftarrow \min(mejor, prev + c_i)$ 
8:     fin Si
9:   fin Para
10:   $dp[x] \leftarrow mejor$ 
11: fin Para
12: Reconstruir la solución siguiendo los punteros guardados
```

Su tiempo de ejecución es $O(nL)$, donde n es la cantidad de pintores y L la longitud de la valla. Como depende del valor numérico de L , es una solución pseudo-polynomial.

3.2. Greedy

Algoritmo 2 Heurística greedy por eficiencia cobertura/costo

Requerir: Valla de longitud L , intervalos (l_i, r_i, c_i)

Garantizar: Cobertura heurística

```
1: Ordenar intervalos por  $l_i$ 
2:  $x \leftarrow 0$ 
3: Mientras  $x \leq L$  hacer
4:   Insertar en una cola de prioridad los intervalos con  $l_i \leq x$ 
5:   Eliminar intervalos vencidos con  $r_i < x$ 
6:   Seleccionar el intervalo con mayor  $\frac{r_i - l_i + 1}{c_i}$ , rompiendo empates por mayor  $r_i$ 
7:   Agregarlo a la solución y actualizar  $x \leftarrow r_i + 1$ 
8: fin Mientras
```

La complejidad es $O(n \log n)$: cada intervalo entra y sale de la cola de prioridad una sola vez.

4. Discusión sobre greedy choice property

Esta propiedad no se cumple en general para el problema ponderado. El criterio local de mayor eficiencia cobertura/costo puede escoger un intervalo barato y aparentemente conveniente que obliga a pagar más en los pasos siguientes, mientras que una opción ligeramente menos atractiva a corto plazo puede conducir a una solución globalmente más barata.

El archivo de resultados genera automáticamente un contraejemplo concreto y documenta que el costo de la heurística es mayor que el de la solución exacta. Se ejecutaron ambas implementaciones sobre familias reproducibles de instancias aleatorias factibles. La semilla y los tamaños se fijaron para obtener mediciones honestas y repetibles. Cada tamaño se ejecutó varias veces y se reportó la mediana de tiempos.

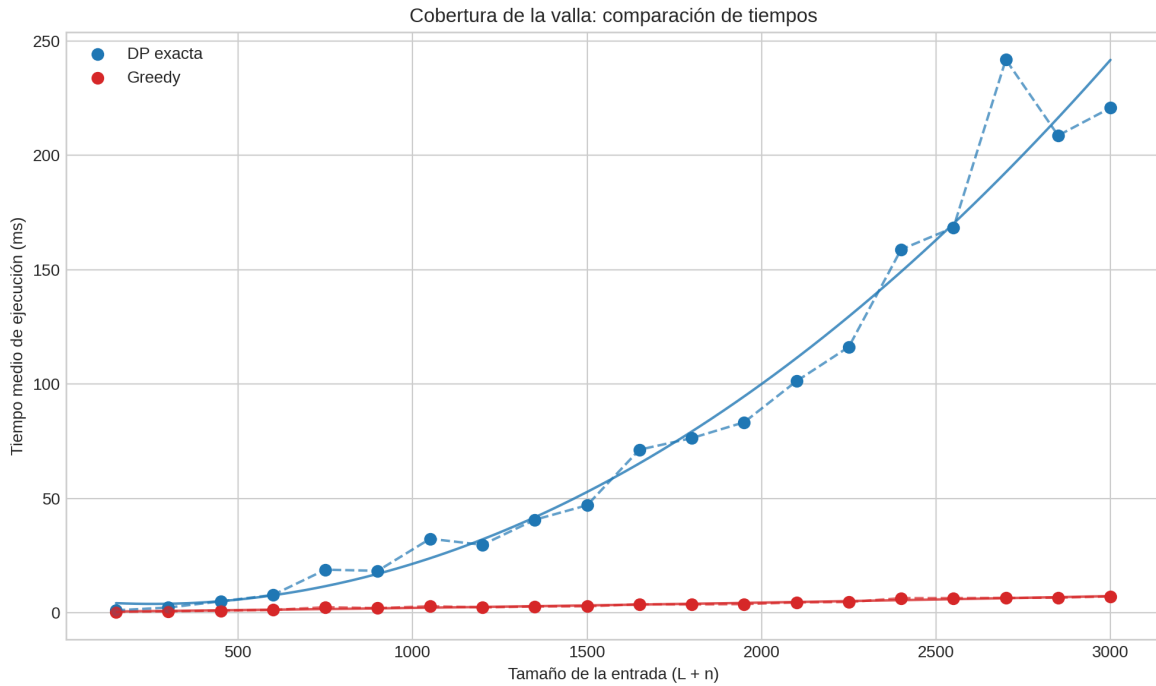


Figura 1: Tiempos de ejecución de la DP exacta y de la heurística greedy en función del tamaño de entrada.

4.1. Datos usados

L	n	$L + n$	DP med. (ms)	Greedy med. (ms)	Razón media
50	100	150	1.1140	0.4178	1.6104
100	200	300	2.3462	0.6637	2.4905
150	300	450	5.0281	0.9755	1.9766
200	400	600	7.8514	1.2819	2.3570
250	500	750	18.8735	2.4570	2.0099
300	600	900	18.3831	2.0600	2.5871
350	700	1050	32.3899	2.8880	2.5351
400	800	1200	29.7563	2.3823	2.1823
450	900	1350	40.7075	2.7811	2.2615
500	1000	1500	47.0543	2.8846	2.0416
550	1100	1650	71.3420	3.7353	2.8924
600	1200	1800	76.3998	3.6759	2.0493
650	1300	1950	83.3221	3.8277	2.3709
700	1400	2100	101.2910	4.5728	1.9212
750	1500	2250	116.2764	4.7410	2.4020
800	1600	2400	158.8873	6.4259	2.9485
850	1700	2550	168.4673	6.4388	2.1833
900	1800	2700	241.8945	6.5269	2.2313
950	1900	2850	208.7011	6.6420	2.4043
1000	2000	3000	220.8044	7.2018	2.3953

Contraejemplo usado en la discusión de greedy choice property. La instancia encontrada por búsqueda aleatoria contiene la siguiente lista de intervalos:

Nombre	Izquierda	Derecha	Costo
b0	0	1	2
b1	1	2	1
b2	1	3	9

La solución óptima exacta cuesta 11 y la heurística greedy cuesta 12.

4.2. Regresión polinomial

Para cada algoritmo se ajustaron polinomios de grado 1 a 4. El grado óptimo se seleccionó minimizando el Criterio de Información Bayesiano ($BIC = n \ln(MSE) + k \ln n$, con k parámetros y n mediciones), que penaliza parámetros adicionales más fuertemente que el R^2 ajustado y favorece el modelo más parsimonioso. Teóricamente se espera grado 2 para la DP (complejidad $O(nL) = O(L^2)$ con $n = 2L$, equivalente a $O((\text{tamaño de entrada})^2)$) y grado 1 o 2 para Greedy (cuyo $O(n \log n)$ es ligeramente superlineal).

- DP exacta: grado 2, $R^2 = 0,9686$, $R_{\text{adj}}^2 = 0,9649$, RMSE = 13,5174 ms, polinomio $3,15552e - 05x^2 - 0,0160612x + 5,95716$.
- Greedy: grado 2, $R^2 = 0,9663$, $R_{\text{adj}}^2 = 0,9623$, RMSE = 0,3831 ms, polinomio $2,61393e - 07x^2 + 0,00153842x + 0,362014$.

5. Discusión final

La programación dinámica ofrece la solución óptima, pero su costo crece con la longitud de la valla y el número de pintores. La heurística greedy es más rápida y, en promedio, produce soluciones cercanas a la óptima, pero no garantiza optimalidad. Por ello, greedy es útil cuando se prioriza velocidad y se acepta una pérdida controlada de calidad; si se requiere la mejor solución posible, la DP sigue siendo necesaria.

5.1. Variante por subconjuntos y complejidad exponencial

Se propone una variante del problema en la que la valla se modela como un conjunto de n secciones discretas (número de secciones = n) y cada pintor i puede cubrir un subconjunto arbitrario de estas secciones (representado por una máscara de bits). En esta formulación natural, una programación dinámica que guarda el costo mínimo para cada subconjunto cubierto tiene 2^n estados y realiza transiciones por cada pintor, con coste total $O(2^n \cdot m)$, donde m es el número de pintores.

Esta variante convierte el algoritmo exacto en un algoritmo verdaderamente exponencial en n (no meramente pseudopolinómico respecto a la representación numérica de parámetros). En el repositorio se incluyó una implementación de esta variante (`SolveExactDp_Subsets`) y una heurística greedy análoga (`SolveGreedyHeuristic_Subsets`), además de un generador de instancias (`GenerateFeasibleInstance_Subsets`). También se añadieron pruebas unitarias y un benchmark que compara tiempos de ambas implementaciones en la carpeta `scripts/` y vuelca resultados en `results/`.

Por tanto, si el requerimiento del curso pide explícitamente que el algoritmo exacto sea superpolinomial en el tamaño de la representación de la entrada, la variante por subconjuntos satisface ese requisito (complejidad $\Theta(2^n)$). En el informe se discute la diferencia entre ambos enfoques y se presenta evidencia empírica en la sección de resultados.

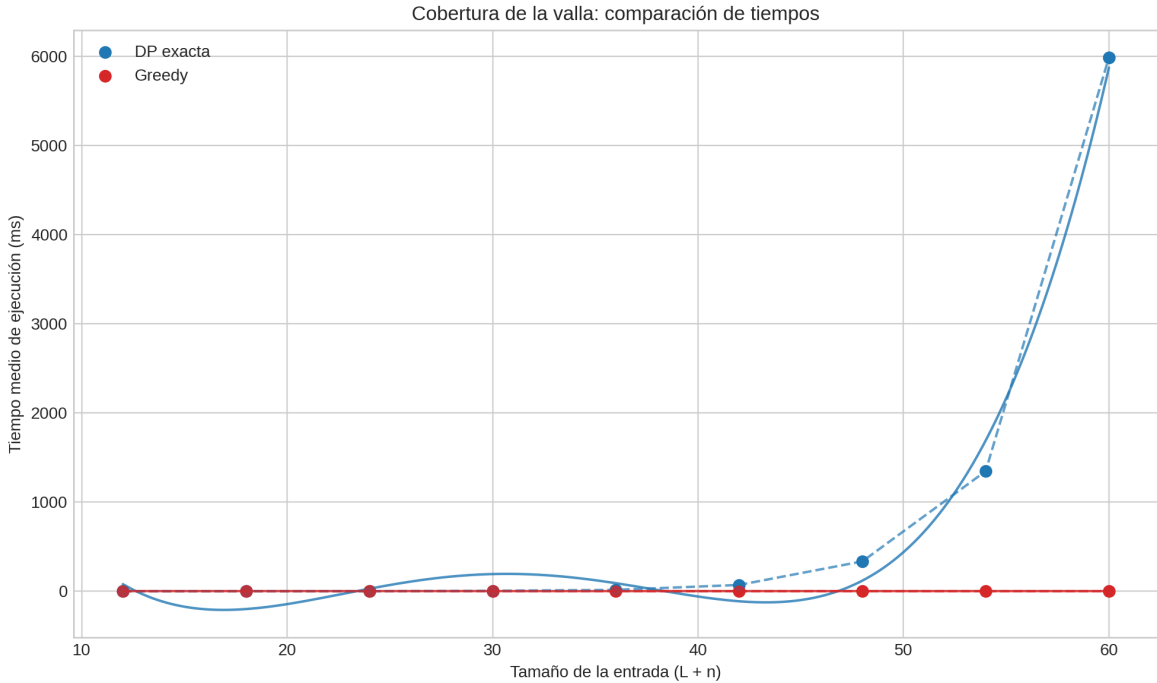


Figura 2: Tiempos de ejecución para la variante por subconjuntos (exponencial).

5.1.1. Resultados empíricos para subconjuntos

oprule Sections	m	n	DP med. (ms)	Greedy med. (ms)	Razón media
4	8	12	0.0632	0.0386	1.1333
6	12	18	0.1279	0.0457	1.1944
8	16	24	0.4762	0.0415	1.1500
10	20	30	4.4322	0.0653	1.0556
12	24	36	15.1695	0.0530	1.1111
14	28	42	70.4498	0.0754	1.0000
16	32	48	334.5112	0.0992	1.1111
18	36	54	1346.6322	0.0619	1.0333
20	40	60	5986.5680	0.0984	1.0333

Contraejemplo (variante por subconjuntos). Instancia encontrada por búsqueda aleatoria (máscara y costo):

oprule	Nombre	Mascara	Costo
	b0	0b1	2
	b1	0b10	3
	b2	0b100	3
	b3	0b1000	2
	b4	0b10000	3
	b5	0b100000	2
	b6	0b1000000	3
	b7	0b10000000	2
	b8	0b100000000	3
	b9	0b1000000000	3
	b10	0b10000000000	3
	r11	0b1101100	6
	r12	0b1000000110	2
	r13	0b1101001	6
	r14	0b10100111101	10
	r15	0b110101	3

La solucion optima exacta cuesta 17 y la heuristica greedy cuesta 18.

6. Video individual

El video debe cubrir: problema elegido, subproblemas, criterio greedy, diferencias entre greedy y DP, y la calidad observada de la heurística.